Francois-Xavier Bagnoud Building
1320 Beal Ave, Ann Arbor, MI 48109

Group 9 Final Project Report

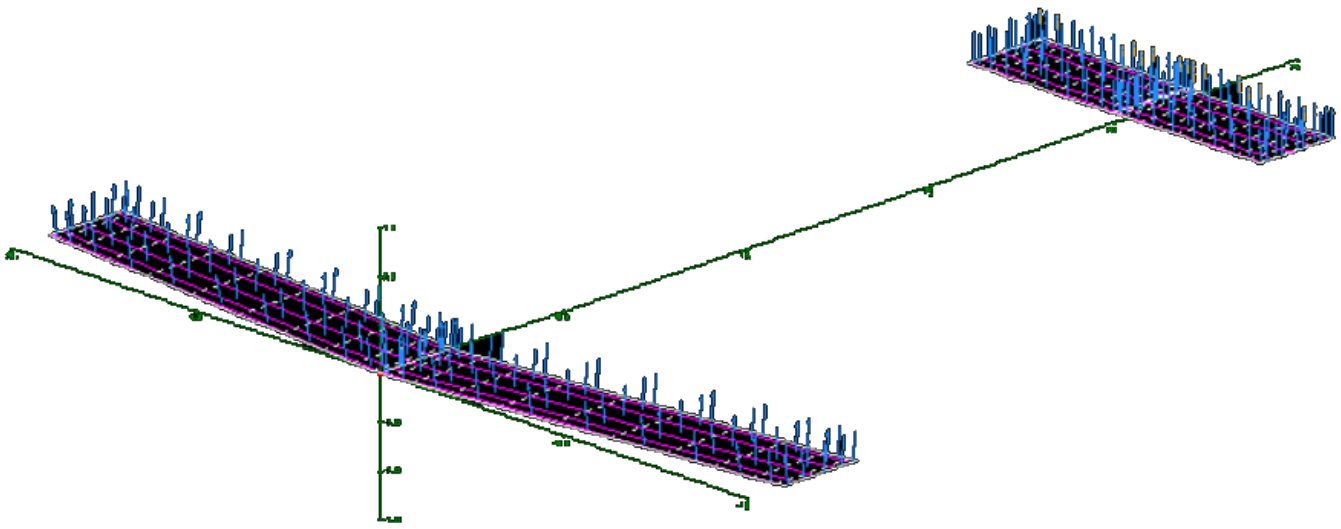
Anthony Argeropoulos, Lucianna Lavey,

Scott Whitman, Klodiana Rexhaj, Gavriil Stavrakas

University of Michigan

Aerospace 325

25 April 2025



Introduction

Foreword

As a final component of Aerospace 325 at University of Michigan, our team has been tasked to develop a first cut design and simulation of a new aircraft aimed to increase strategic airlift capabilities of the United States Air Force. In order to fulfill the design constraints of this assignment, our group relied on knowledge gained during the course of Aerospace 325, as well as Aerospace 201, and significant external research to present a working aircraft design that fulfills all requirements in desired performance and constraints.

Design Constraints

Our design constraints are listed in Table 1 below. While the overall size and mass of the aircraft has been constrained to an upper limit, we are afforded the freedom to design any aircraft within the parameters so long as it meets the desired performance requirements.

Maximum Wing Span	43.672 m
Number of Engines	2 or 4
Dry Weight (No Engine)	30,390 kg
Maximum Takeoff Weight	Minimum 113,400 kg
Fuselage Diameter	4.2672 m
Fuselage Length	34.1376 m
Minimum Cargo Capacity	20,412 kg
Minimum Range	3,380 km
Top Speed	Minimum 179 m/s at 6.7 km
Max Takeoff Speed	71 m/s

Table 1: List of criteria requiring specific wing and fuselage dimensions and aircraft capabilities.

Plane Geometry & Specifications

The following specifications were selected to fulfill the requirements while maintaining optimal design. While more performance can be had within the size and weight requirements, we feel this design, well within the maximum size, most closely fits the desired performance metrics. The dimensions of the wing were adjusted to create a lift profile that meets desired characteristics in takeoff, landing, and steady level flight at the prescribed altitude, while maintaining reasonable flap and elevator deflections.

Our aircraft is capable of flying both faster and higher than the specified minimum requirements. More simulation and refinement should be performed during a second pass refinement of the aircraft to prescribe maximum performance. For this first cut, we merely aim to meet the requirements with performance on the table. See below table for aircraft visual geometry.

Parameter	Wing	Tail	Rudder
Chord length	2 m (constant)	2 m (constant)	2 m (constant)
Span	20 m	8 m	4 m
Sweep	1m	0 m	0 m
NACA airfoil	4412	0014	0014
Control Surfaces	0.6 m (flap)	0.4 m (elevator)	N/A

Additional Parameters

Fuselage length	34.1376 m	Tip to Center Mass	12.1376 m
Fuselage diameter	4.2672 m	Center Mass to Tail	22 m
Neutral Point	4.5 m - 4.7 m	Tip to Neutral Point	16.63 m - 16.83 m

Table 2: Wing Characteristics [Appendix: 29 & 30]

Engine Selection

For our aircraft we are utilizing four Rolls-Royce AE 2100D3 Turboprop engines. These engines are well within the performance requirement, already in the Air Force's arsenal on the C130-J, and have enough performance remaining for significantly increased performance over the prescribed requirements.

Quantity	4
Type	Rolls-Royce AE 2100D3 turboprop
Dry Weight	806 kg
Fuel type	JP-8
Shaft Power	3,410 kW
Specific Fuel Consumption	$7.77*10^{-8}\ \frac{kg}{W*s}$
Propeller Efficiency	0.65

Table 3: Specifications for the AE 2100D3 turboprop [Reference #1].

Configuration & Performance

During this section we describe both the required performance metrics and the configuration we used with our design to meet those specifications. For this design, we evaluated Takeoff and Landing configurations at sea level, as well as cruise at the prescribed flight level. For all stages, we assume maximum takeoff weight, at 113,400kg, including for landing conditions.

Takeoff

Our takeoff condition assumes sea level, standard atmospheric conditions.

Takeoff Configuration

Our take off configuration was determined by constraining alpha and varying flap and elevator deflection to achieve the desired lift. This given configuration proves the plane is able to fly with given requirements.

Angle of attack	3 deg
Flap deflection	12.43 deg
Elevator deflection	-3.4707 deg

Table 4: Takeoff Configuration [Appendix #: 11]

Takeoff Performance

The resulting performance shown below is a result of constraint around steady flight when weight is equal to lift.

Parameter	Proposed Aircraft	Requirement
Take off Speed	65 m/s	Maximum 75 m/s
Mach	0.1910	– – –
Lift/Weight	1.00 (steady flight)	1.00
Max Takeoff Weight	115,666 kg	Minimum 113,400 kg
Vstall	52.1054 m/s	– – –
Point Moment	0	Must equal zero
Statically Stable	Yes	Must Satisfy
Neutral point	4.68 m downstream of leading edge of wing	Must conduce restorative stability for pitch up
Max Thrust	136400 N	Thrust > Drag
Drag	3964.77 N	– – –
Lift	116000 N	– – –

Table 5: Takeoff Performance [Appendix #: 17]

Cruise (6705.6 m)

While our aircraft is capable of flying higher and faster, we have selected this altitude to match the requirements for comparison. The aircraft's engines have performance on the table compared to the drag, and the angle of attack can be increased significantly from its approximately zero value. Flaps also could be used to increase lift.

Cruise Configuration

For cruise the configuration is constrained around the flaps deflection angle being equal to zero. This is because we want a minimal amount of drag.

Angle of attack	-0.6297 deg
Flap deflection	0 deg
Elevator deflection	1.6556 deg

Table 6: Cruise Configuration [Appendix #: 7]

Cruise Performance

The resulting performance is constrained to steady flight while being able to hold a speed of 180 m/s at 6705 km altitude. Flying in these conditions our aircraft exceeds the requirements in all sections.

Parameter	Proposed Aircraft	Requirement
Stability Speed	180 m/s	Minimum 179 m/s
Mach	0.5741	– – –
Lift/Weight	1.00	1
Range	3685.88 km	Minimum 3,380 km
Vstall	93.58 m/s	– – –
Point Moment	0	Equal to zero
Statically Stable	yes	Must Satisfy
Neutral point	4.54 m downstream of leading edge of wing	Must conduce restorative stability for pitch up
Max Thrust	49256 N	Thrust > Drag
Drag	18043.49 N	– – –
Lift	115870 N	– – –

Table 7: Cruise Performance [Appendix #: 18]

Landing

Our landing conditions assume that the aircraft weight is as much as the takeoff weight, and that we are landing at sea level.

Landing Configuration

For landing the configuration is constrained by the angle of attack while the flap and elevator deflection is varied to achieve the desired conditions for landing.

Angle of attack	8 deg
Flap deflection	5.8497 deg
Elevator deflection	-10.0374 deg

Table 8: Landing Configuration [Appendix #: 3]

Landing Performance

The resulting performance shown below is conveying the necessary conditions needed to land such as landing speed targeted at 1.2*Vstall, steady, and statically stable flight. Here we see reasonable numbers across the board for landing conditions.

Parameter	Proposed Aircraft	Requirement
Landing Speed	62.53 m/s	1.2*Vstall
Mach	0.1837	– – –
Lift/Weight	1.00	1
Max Landing Weight	115,666 kg	– – –
Vstall	52.11 m/s	– – –
Point Moment	0	Equal to zero
Statically Stable	yes	Must Satisfy
Neutral Point	4.74 m downstream of leading edge of wing	Must conduce restorative stability for pitch up
Max Thrust	141800 N	Thrust > Drag
Drag	5062.17 N	– – –
Lift	115960 N	– – –

Table 9: Landing Performance [Appendix #: 19]

Assumptions

Compressibility

- Accounting for compressibility in coefficient of lift and coefficient of drag

Flat Plate Theory

- While calculating the drag for the fuselage

Lift

- Negligible lift effect from fuselage

Structural Assumption

- Center of mass anywhere and not affected by fuel depletion
- All structure accounted for in dry weight

Propeller efficiency

- Assuming 65% propeller efficiency based off research
- Assuming fuel consumption of $7.77*10^{-8}\ \frac{kg}{W*s}$ based off research

Drag

- No change drag while landing due landing gear deployment

Assumptions Given in Rubric

- Fly be wire for stability control
- Area of one elevator and rudder are equal

Methods

Iterative Design Process

When creating the design, we looked at many factors to best fit an aircraft to the requirements while maintaining minimal assumptions. We started by comparing to other aircraft with similar performance, which we found from the Air Force's C130-J. With that starting point, we designed a maximally effective aircraft within the maximum size and weight. After finding that the aircraft was overly performant, we trimmed the design back until the performance metrics more closely match that of the design requirements. This was done via a process of creating the aircraft in AVL, running attached custom designed matlab scripts (attached in the appendix) to find the required performance metrics at different stages of flight, and then going back to AVL to attempt to find an alpha, flap deflection, and elevator deflection for stability under the condition. This process was done iteratively until a satisfactory design was found.

After the design was mostly defined, we conducted further analysis in XFOIL, for drag and maximum lift configurations, and conducted further matlab analysis of the design to determine if the aircraft meets all other requirements, which the final design does.

Lift Calculation

To calculate the lift performance of the aircraft across varias flight conditions. First we used custom matlab functions to calculate what our required lift performance would be. We then set the aircraft in AVL, since we knew what Cl we would need in order to produce the lift we wanted, we constrained either the alpha, or flap deflection to achieve the desired lift, fixing one or the other for reasonable flying conditions. We then varied the elevator deflection to a deflection angle that properly trimmed the aircraft for zero pitch moment.
Landing: [3]
6.7 km Altitude: [7]
Takeoff: [11]

In this process, in matlab, we used the following equation for lift correction for compressibility. The matlab code is appended in the appendix [15]

$$Lift = \frac{\frac{1}{2}\rho U_\infty^2 c_l c}{\sqrt{1-M_\infty^2}} * b$$

Drag Calculation

In order to calculate the drag for our identified aircraft conditions and configuration at takeoff landing and cruise, we calculated the drag of the fuselage, main wing, and tail separately, accounted for compressibility, and added them together for a total drag.
For the fuselage, we used flat plate theory by unrolling the cylinder, taking the top dome of the aircraft and flattening it, assuming more area in the dome section by simply taking the arc length and adding it to the total length, we then integrated Cf assuming turbulent flow due to the speed and altitude calculations giving an Re number greater than 1e6. Using this we created a matlab function that would calculate the drag given input speed, and atmospheric conditions using the following equations:
Appendix Code: [21]

$$c_f = \frac{0.0592}{Re_x^{0.2}} \qquad Drag = \frac{1}{2}\rho U_\infty^2 \int_A c_f(x) dA$$

Then to calculate the drag of the wing and tail, we took our aircraft configuration and flight conditions, used XFOIL to find a coefficient of drag, and then integrated that coefficient of drag over the surface of the wing. Due to our aircraft being of constant chord, our integration simplifies into a simple equation of wing area.
Landing: Wing [5], Tail [6]
6.7 km Altitude: Wing[9], Tail[10]
Takeoff: Wing[13], Tail[14]

$$Drag = \frac{\frac{1}{2}\rho U_\infty^2 c_d c}{\sqrt{1-M_\infty^2}} * b$$

Thrust Calculation

For the thrust calculation, and our assumptions, we were able to make a simple matlab function that would find the thrust given input conditions and configuration. Here we used the following equations:
Appendix Code: [23]

$\boldsymbol{\eta}$ = 0.65

$$T = \frac{\eta * P_s}{U_\infty}$$

Stall Check

For our stall check, we used XFOIL to simulate the airfoil's lift capability over a range of alpha's. XFOIL was used over AVL, because AVL's simulation of high alpha was found to be unreliable and inaccurate. After identifying the alpha for max Cl, we put these findings into AVL. Using AVL we were able to determine a maximum lift condition for both flaps fully extended, and fully retracted. Using these, we were able to check for stall conditions using a custom matlab function that was called for each configuration to either check for stall, or decide on an airspeed for the aircraft, in the case of landing at 1.2 * Vstall.
Appendix Code: [24]

$$V_{stall} = \sqrt{\frac{2*W_i}{\rho*S*C_{L,max}}}$$

Range Calculation

Using the initial maximum takeoff weight, cruise conditions, and assuming full tanks that are depleted all the way, we used matlab with another custom function and the range equation to determine a total range for the aircraft. The following equation was used:
Appendix Code: [25]

$$Range = \frac{U_\infty * W_i}{16 * P_s * c} * ln(\frac{W_i}{W_f})$$

Proofs

XFOIL

XFOIL was used for calculating the drag over the wings, tail and rudder. By integrating the skin friction coefficient over the flat plates we were able to prove that for one wing we get a similar drag calculated by XFOIL. Code for this calculation can be referenced in the appendix. [26]

AVL

AVL was used to calculate the lift characteristics of the wings, and to find stability configurations for varying points in flight. We analyzed AVL using two methods. First we found the zero lift angle of attack for a NACA airfoil, and then we used this airfoil in AVL to create a wing, which we were able to access for zero lift angle. We found that these two methods generated the same result. To prove that AVL can be used for these calculations we compared it with code from homework 7 that used vortex panel method to compute the coefficient of lift. The code from homework and the comparison calculated from AVL can be referenced in the appendix. [28a & 28b]

Compressibility

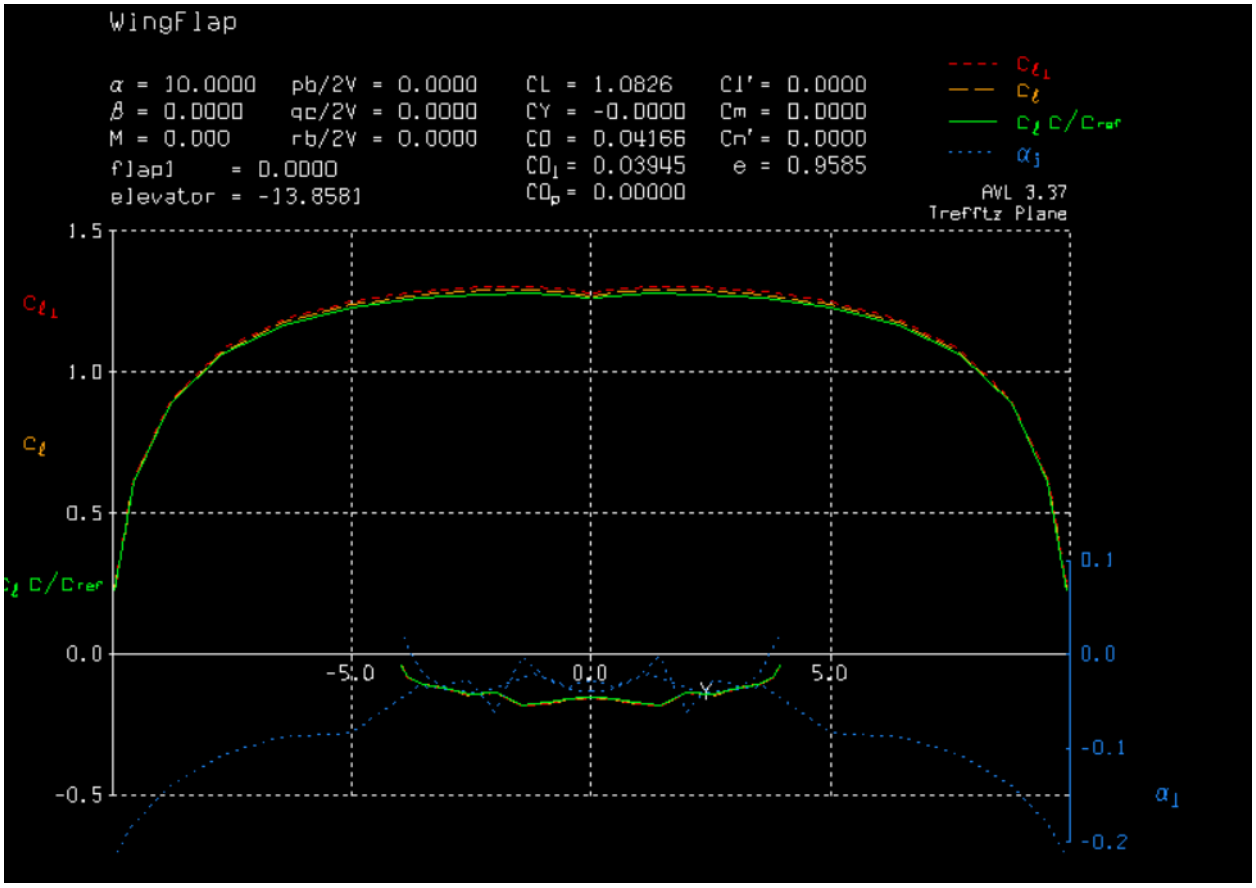
It has been proven in chapter 8.3 of the AEROSP 325 book written by Professor K.Fidkowski that when assuming incompressibility the coefficient of lift and moment coefficient can be multiplied by the compressibility correction factor to account for compressibility. This has not been proven for the coefficient of drag and therefore we must find a method to see if this factor can be applied to account for compressibility. We found that if a given coefficient of drag for incompressibility is multiplied by the compressibility factor for a range of mach numbers it very closely matches drag coefficients given by Mfoil calculations for compressibility. The code for this proof can be found in the appendix. [27]

References

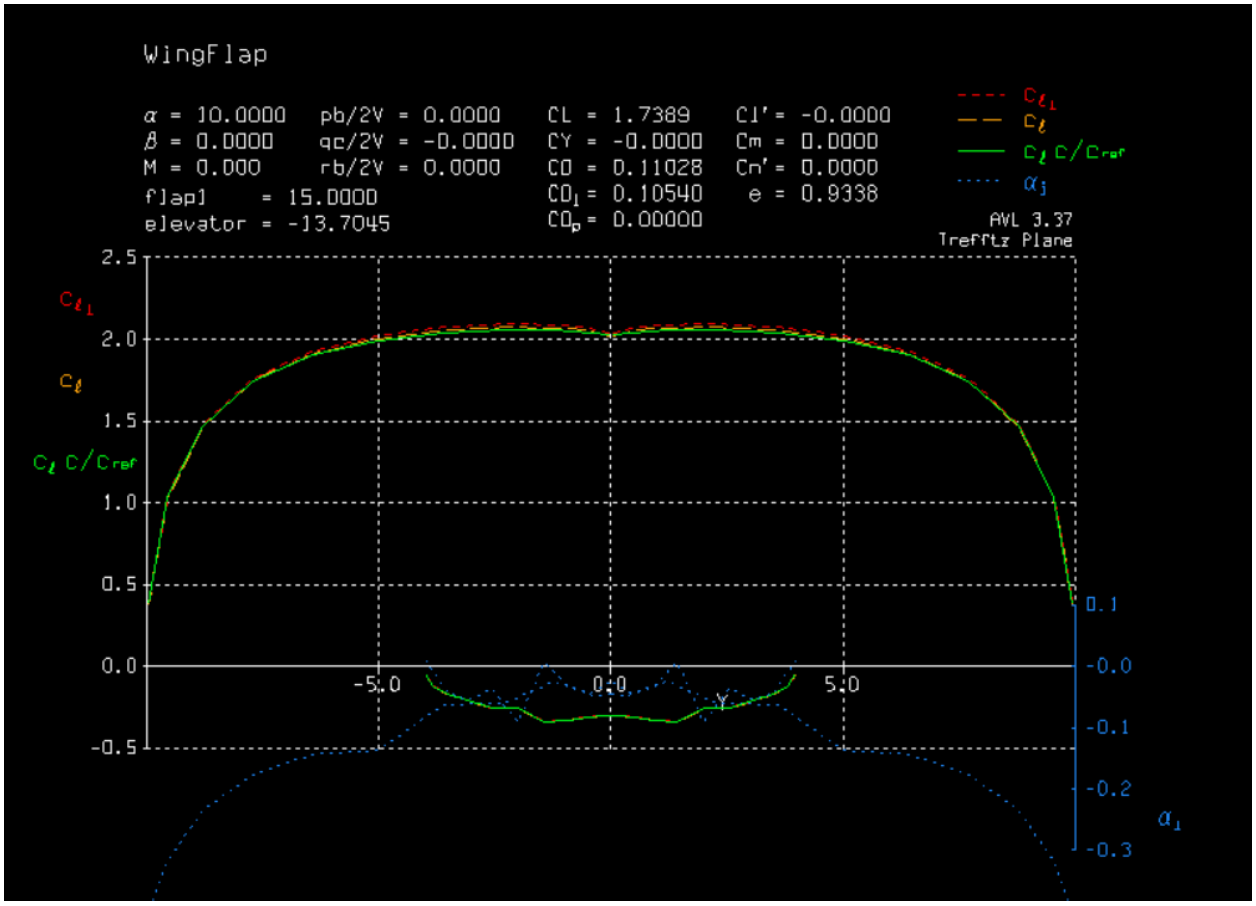
1. Engine: https://www.rolls-royce.com/~/media/Files/R/Rolls-Royce/documents/defence/VCOMB%203855%20Defence%20Data%20sheet%20-%20AE%202100.pdf
2. AVL: https://web.mit.edu/drela/Public/web/AVL/
3. XFOIL: https://web.mit.edu/drela/Public/web/XFOIL/
4. Atmospheric Conditions: https://aerospaceweb.org/design/scripts/atmosphere/
5. Thrust & Efficiency Equations: https://web.mit.edu/16.unified/www/SPRING/systems/Lab_Notes/airpower.pdf
6. 201 book by K. Fidkowski (Vstall & Range)
7. 235 book by K. Fidkowski (Fuselage drag & Compressibility factor)

Appendix

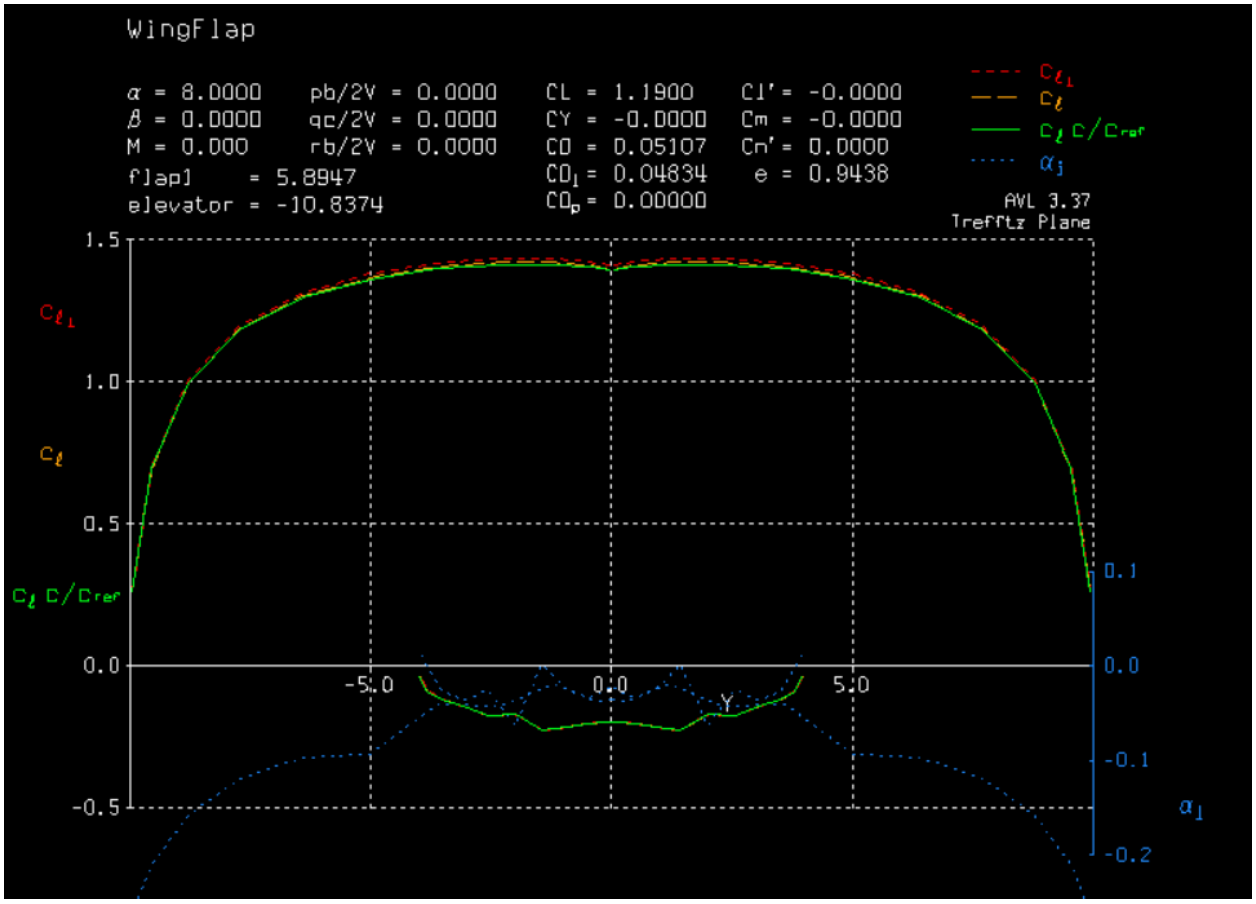
1: Cl Max Without Flaps Simulation



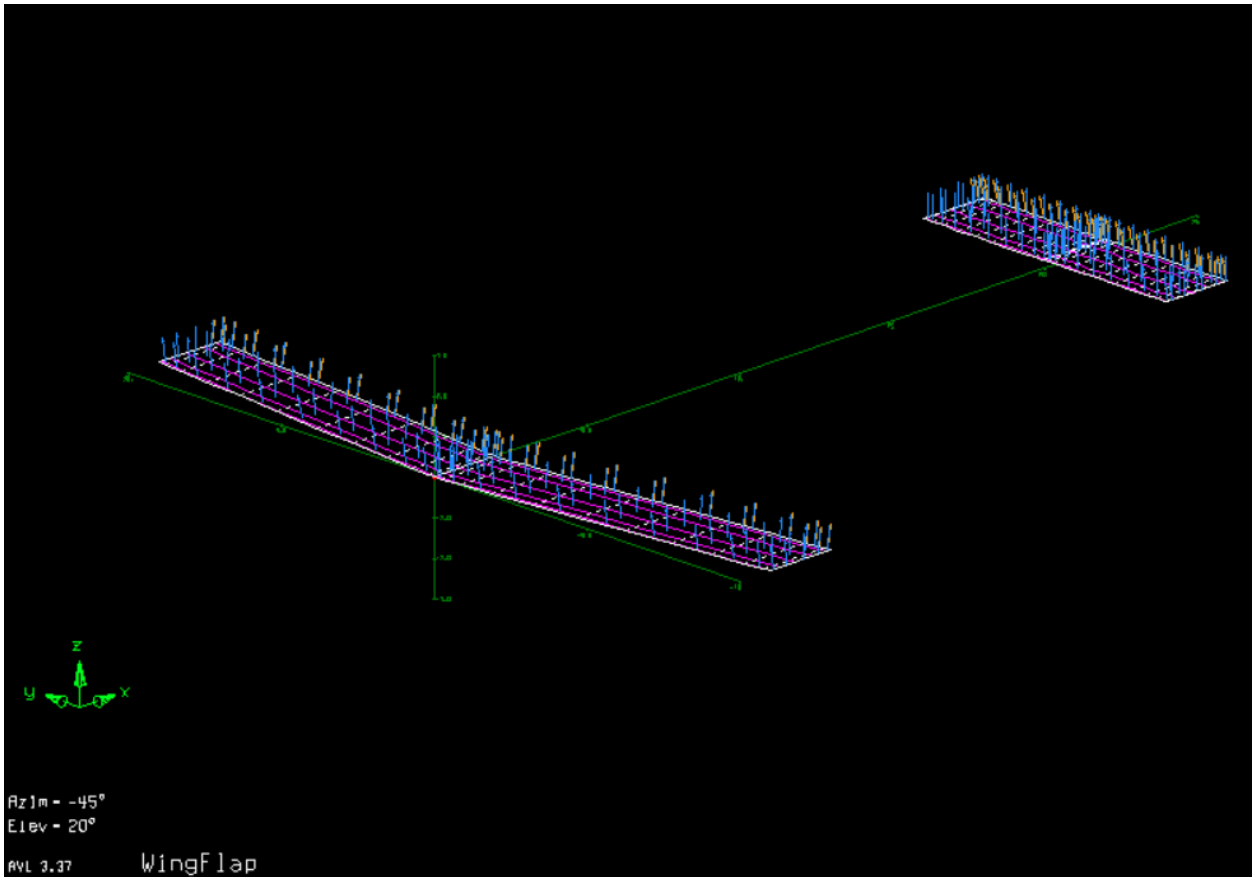
 # 2: Cl Max With Flaps Simulation



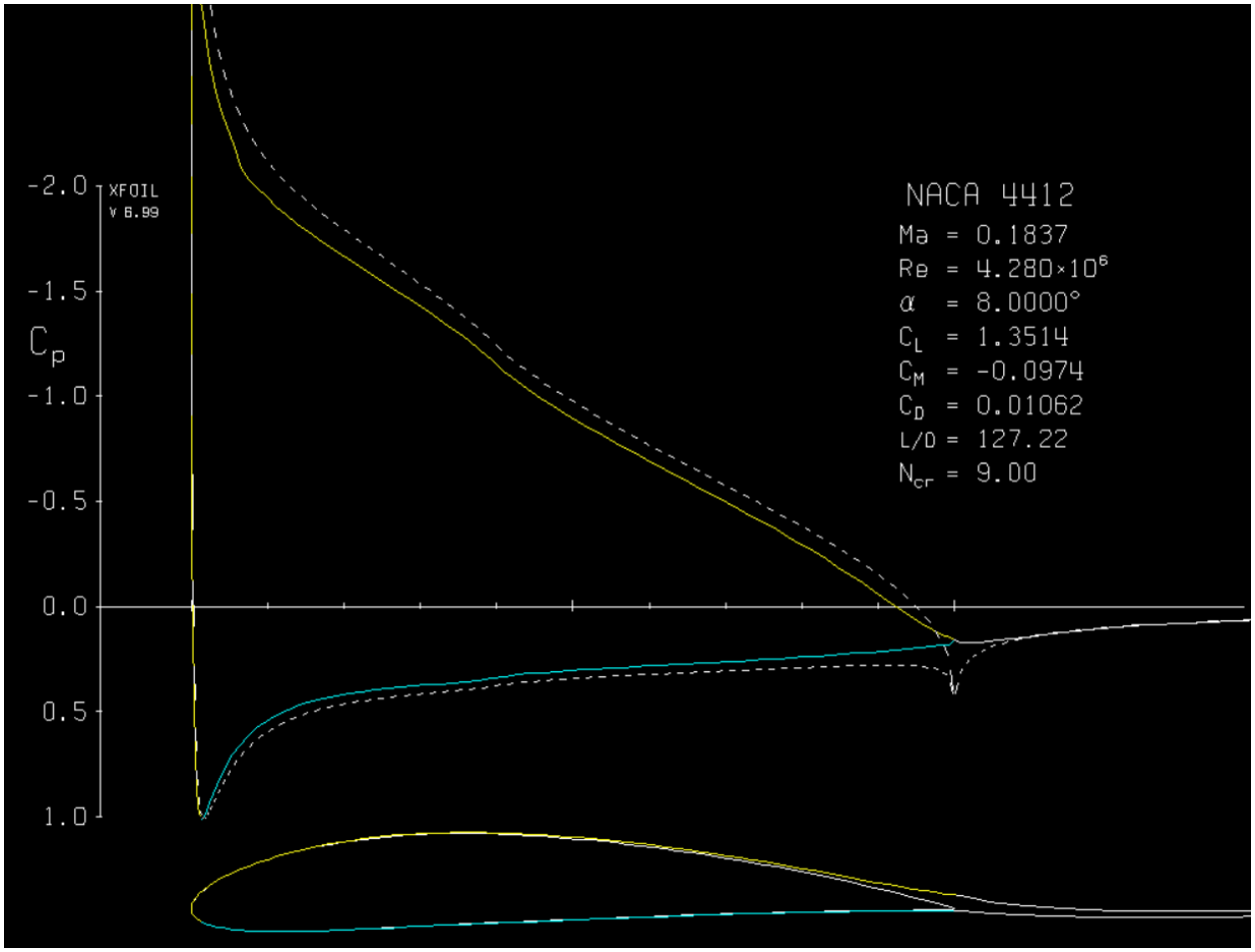
 # 3: Landing Configuration Simulation



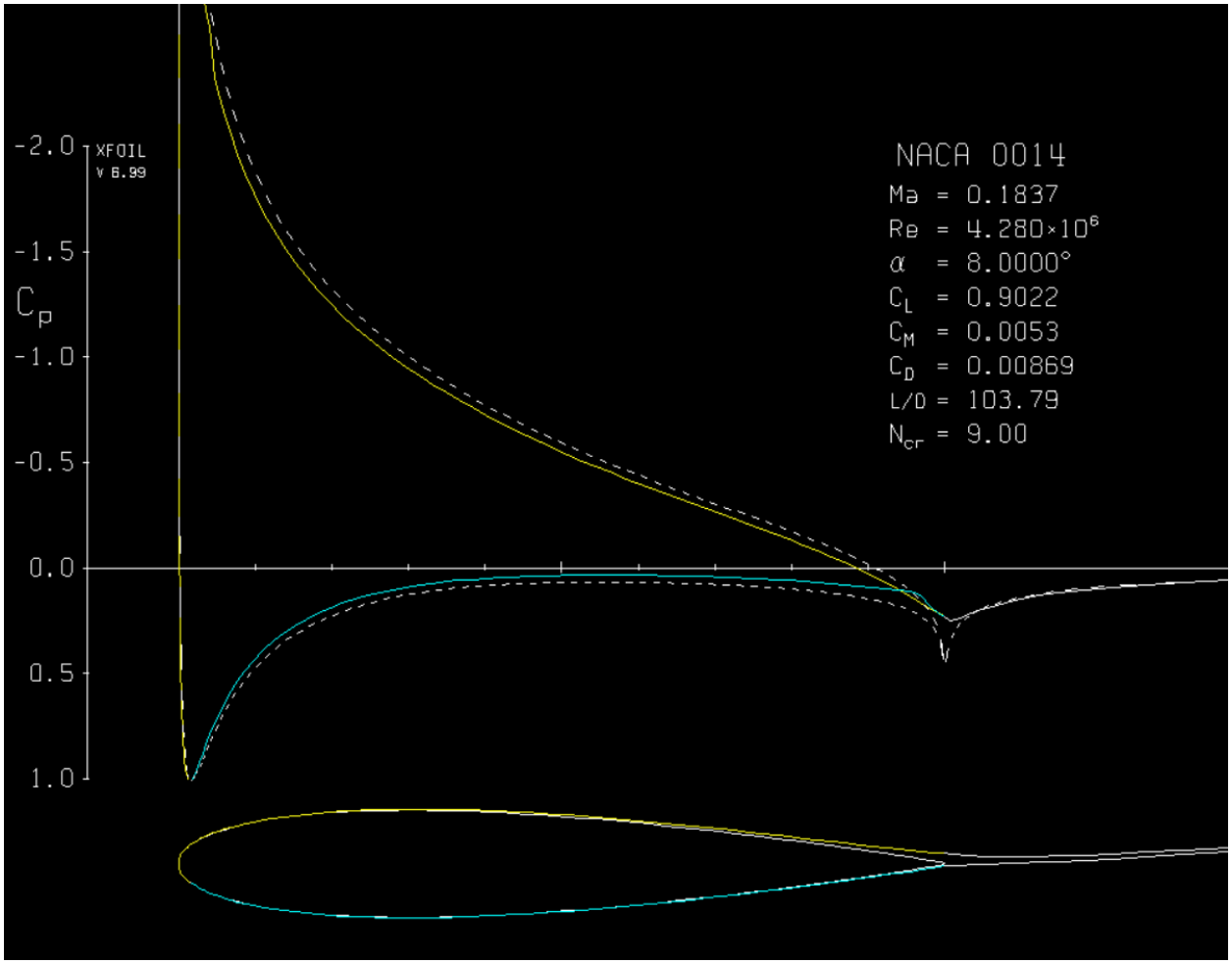
 # 4: Landing Configuration Genometry



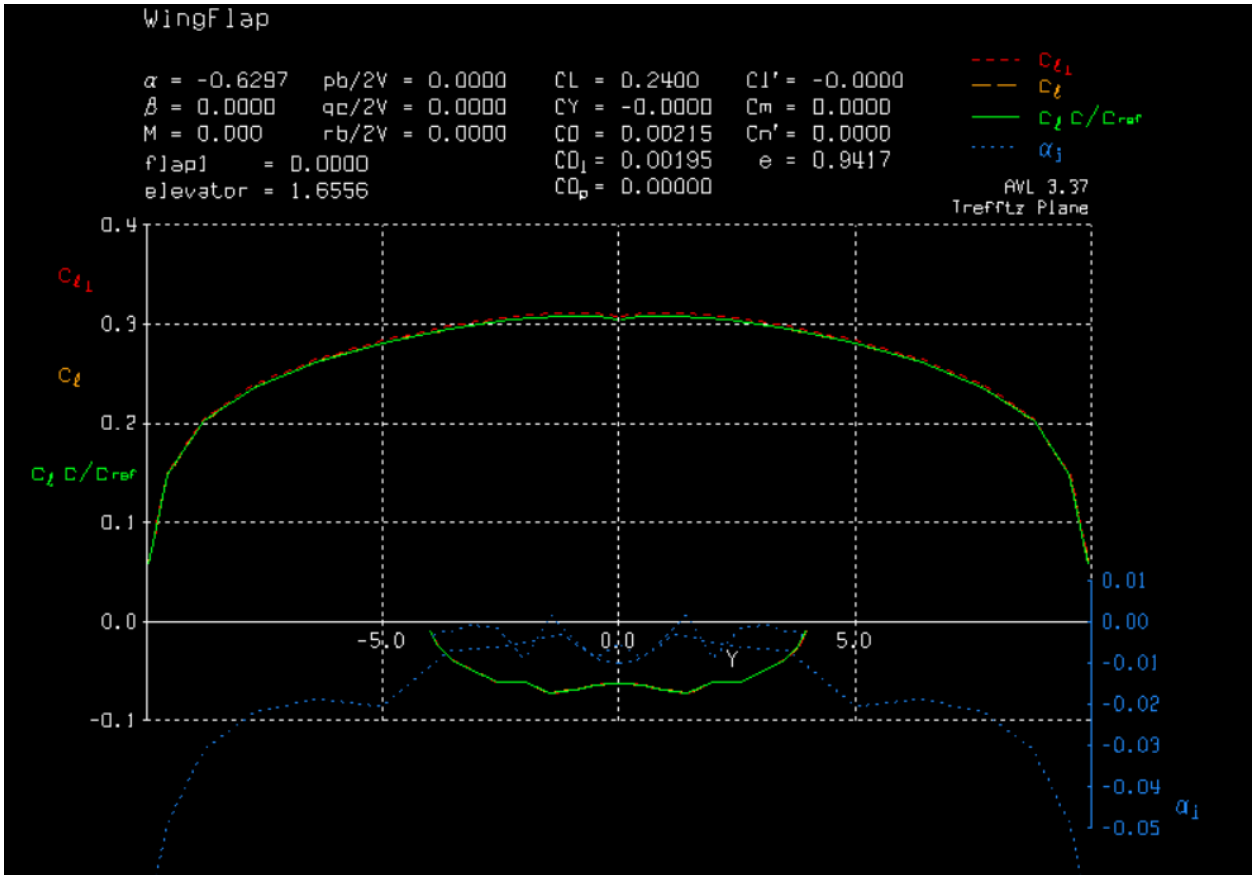
 # 5: Landing Main Airwing C_d Simulation



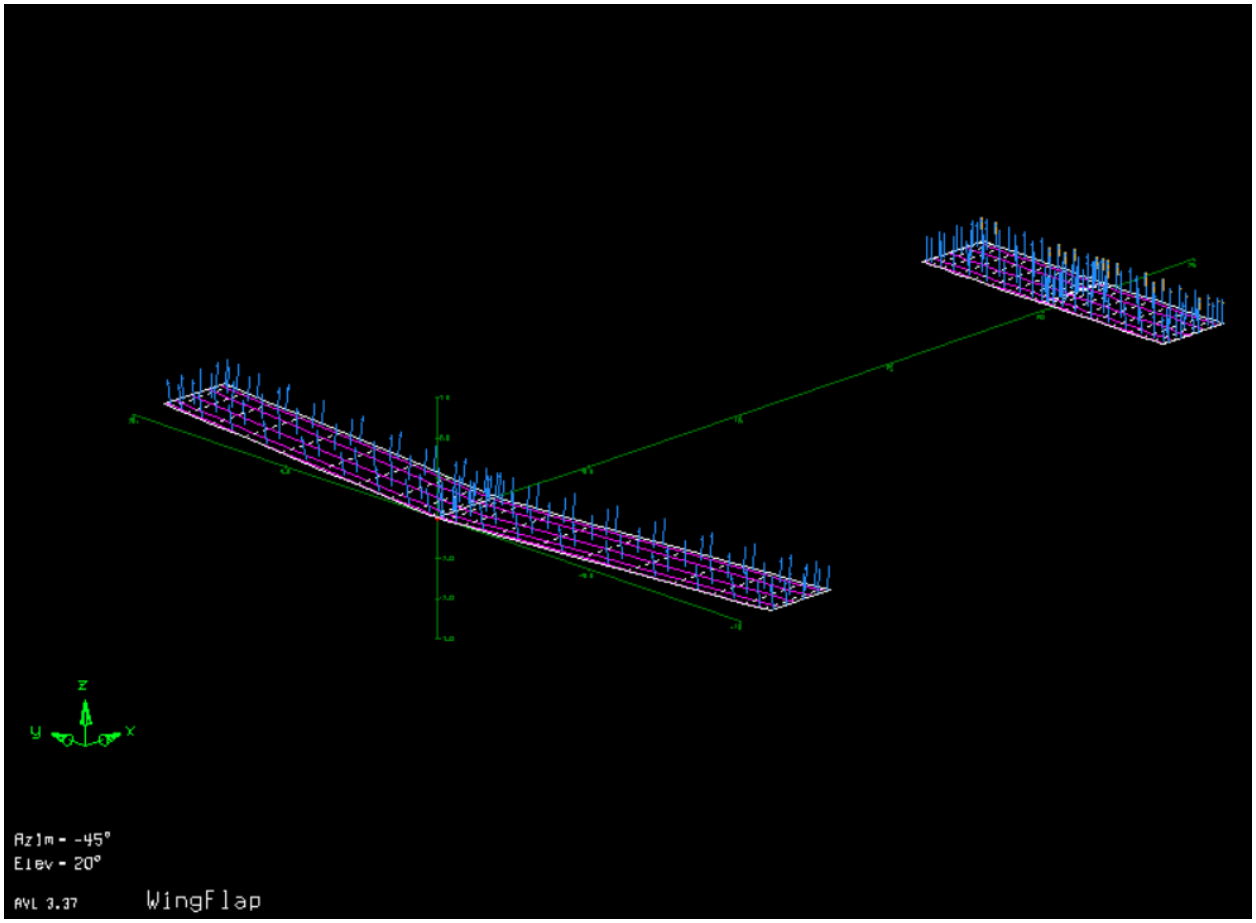
 # 6: Landing Tail Airwing C_d Simulation



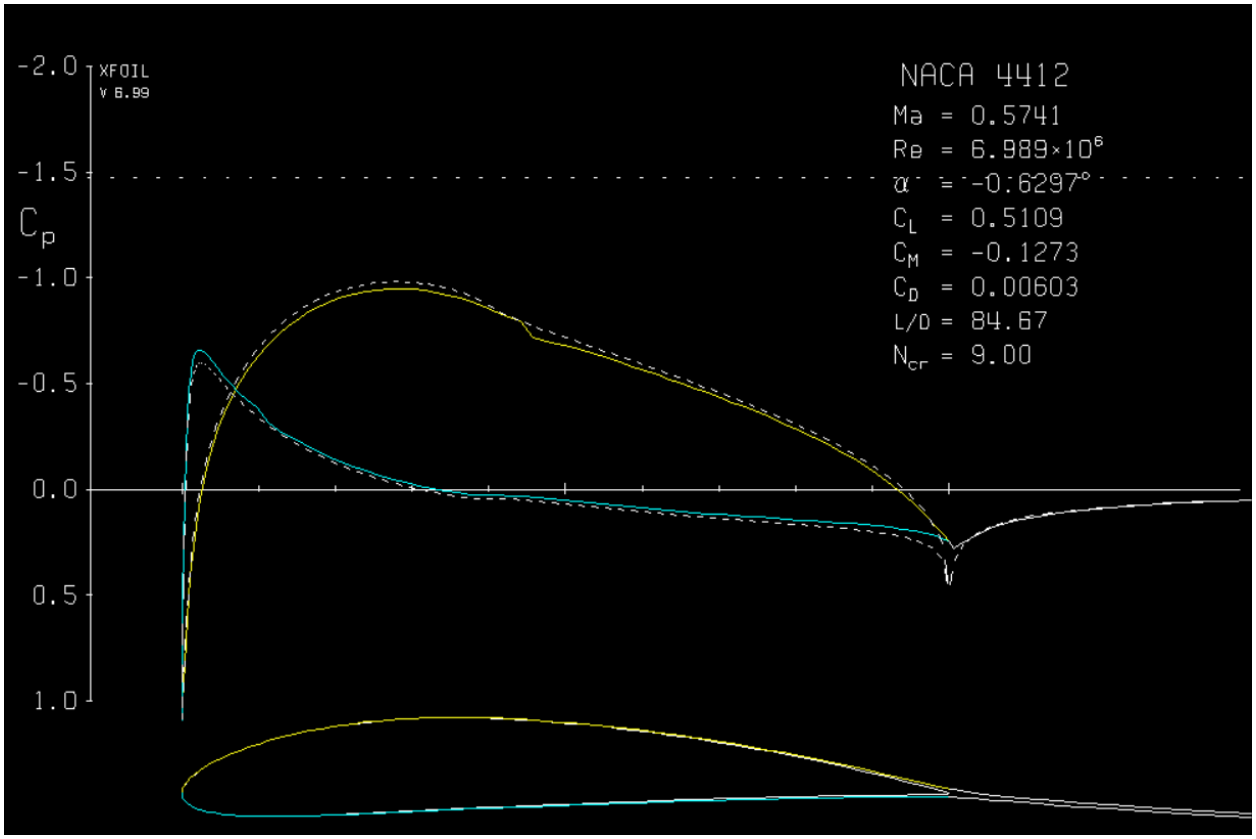
 # 7: 6.7 km Altitude Configuration Simulation



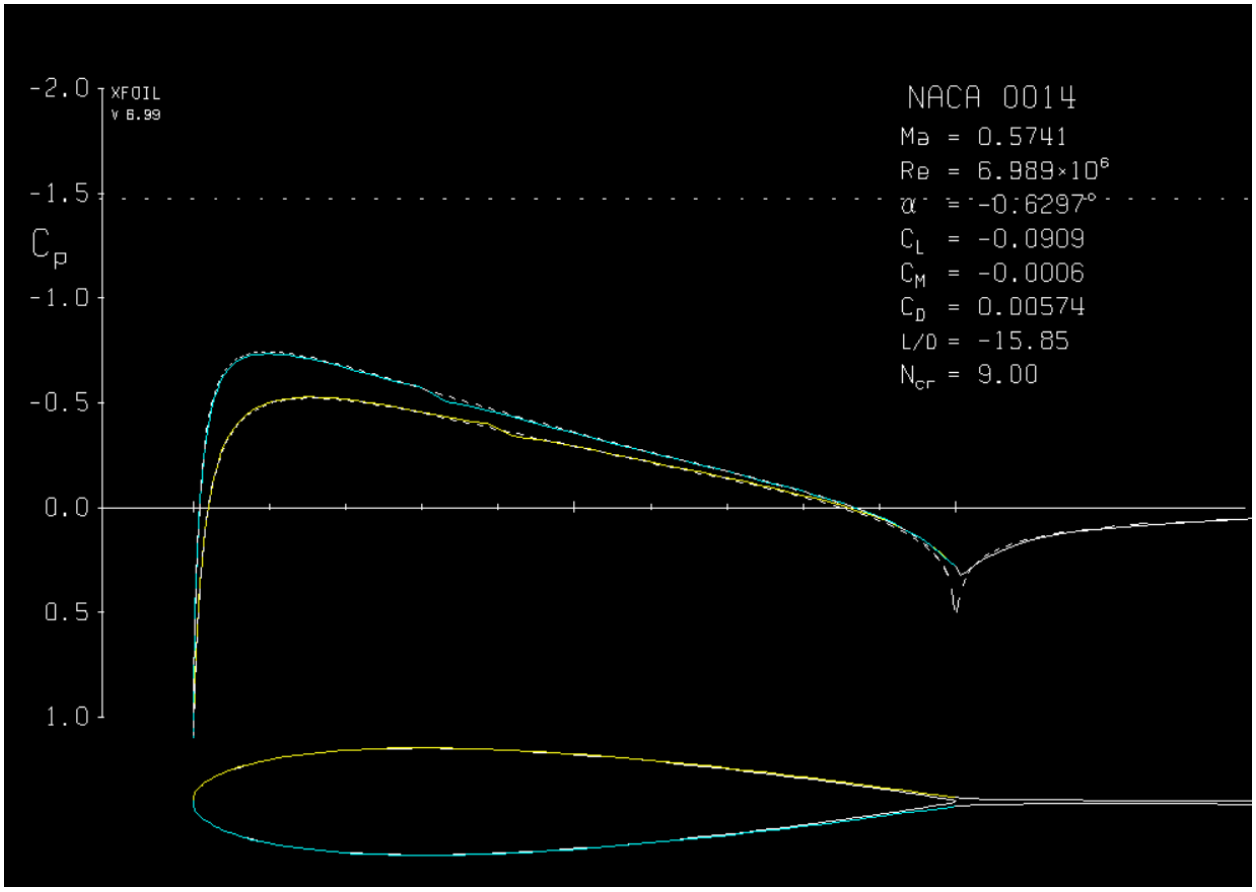
 # 8: 6.7 km Altitude Configuration Genometry



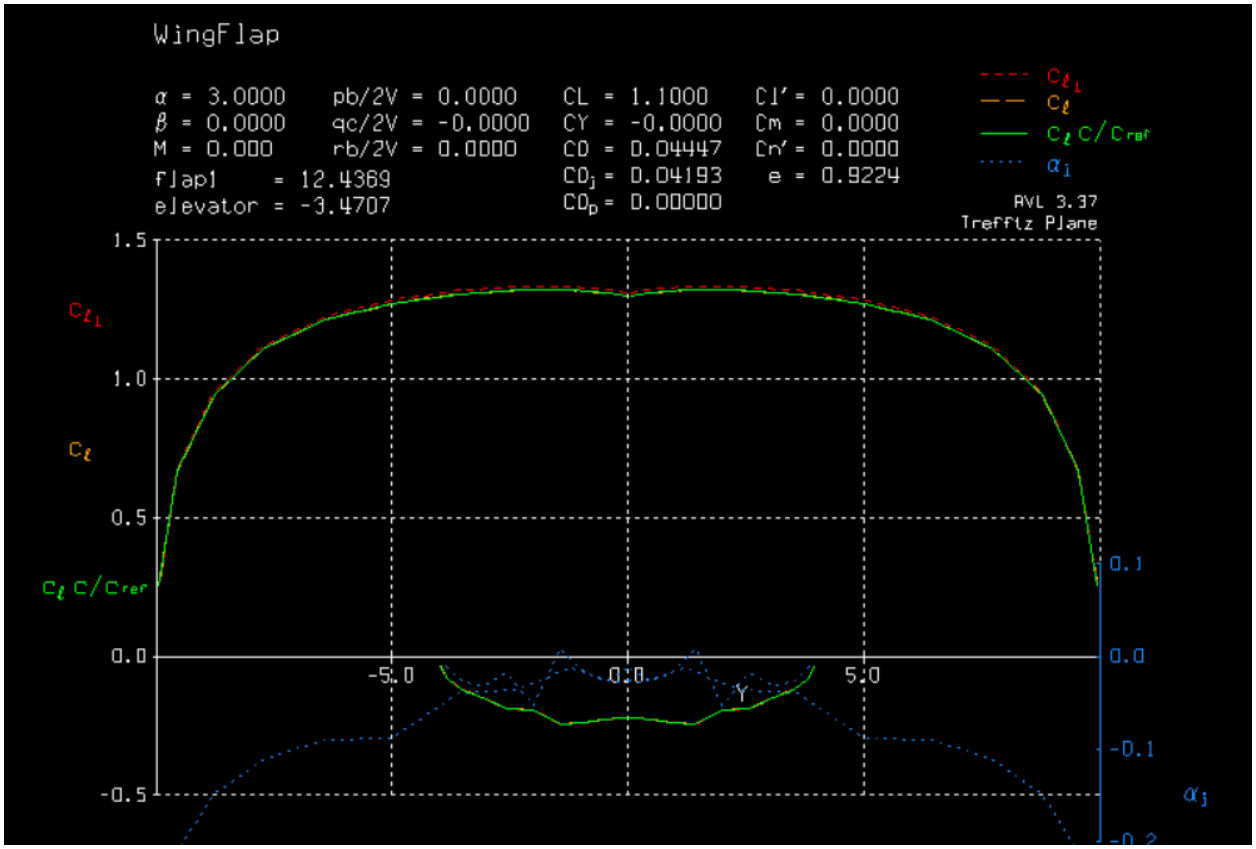
 # 9: 6.7 km Altitude Main Airwing C_d Simulation



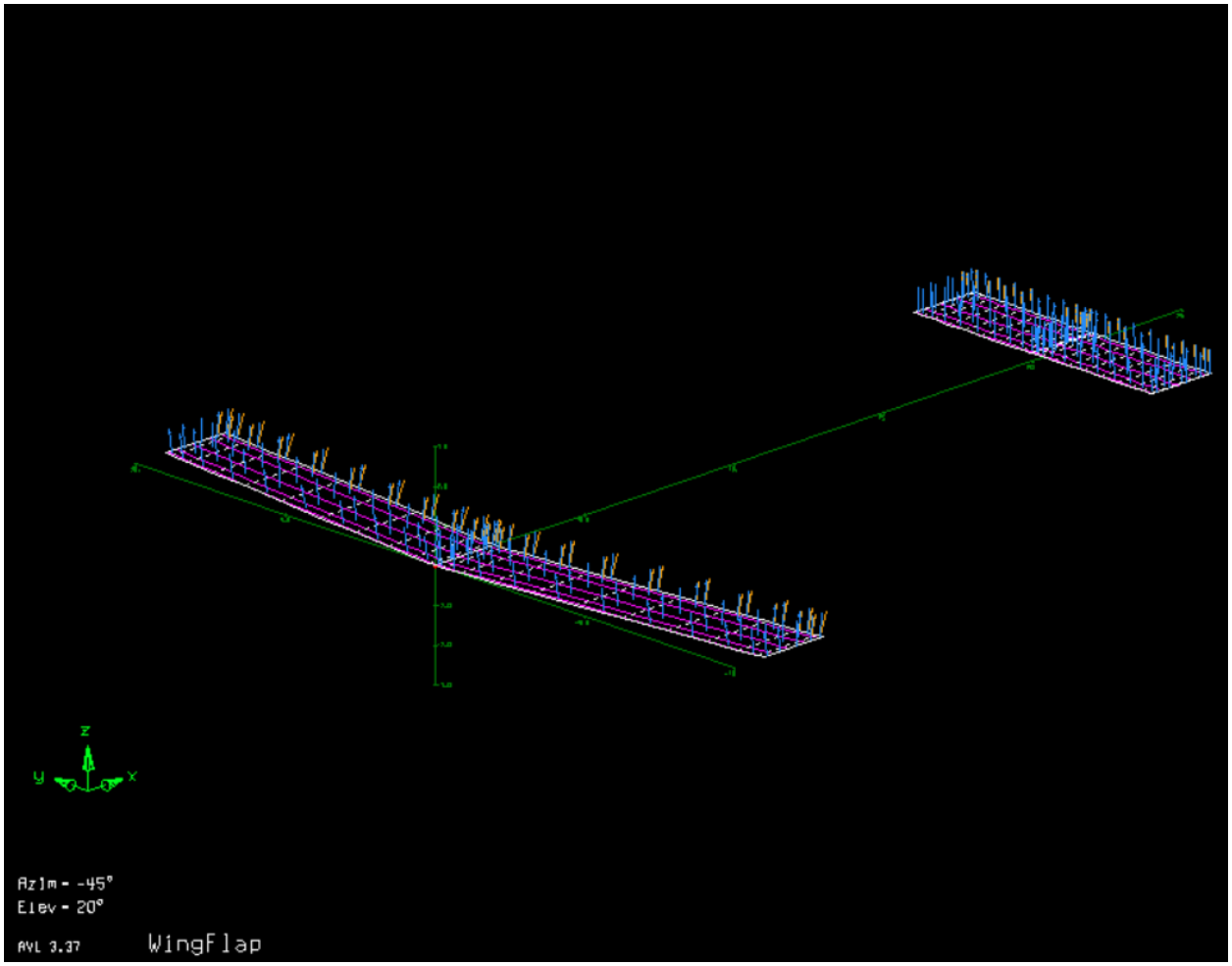
 # 10: 6.7 km Altitude Tail Airwing C_d Simulation



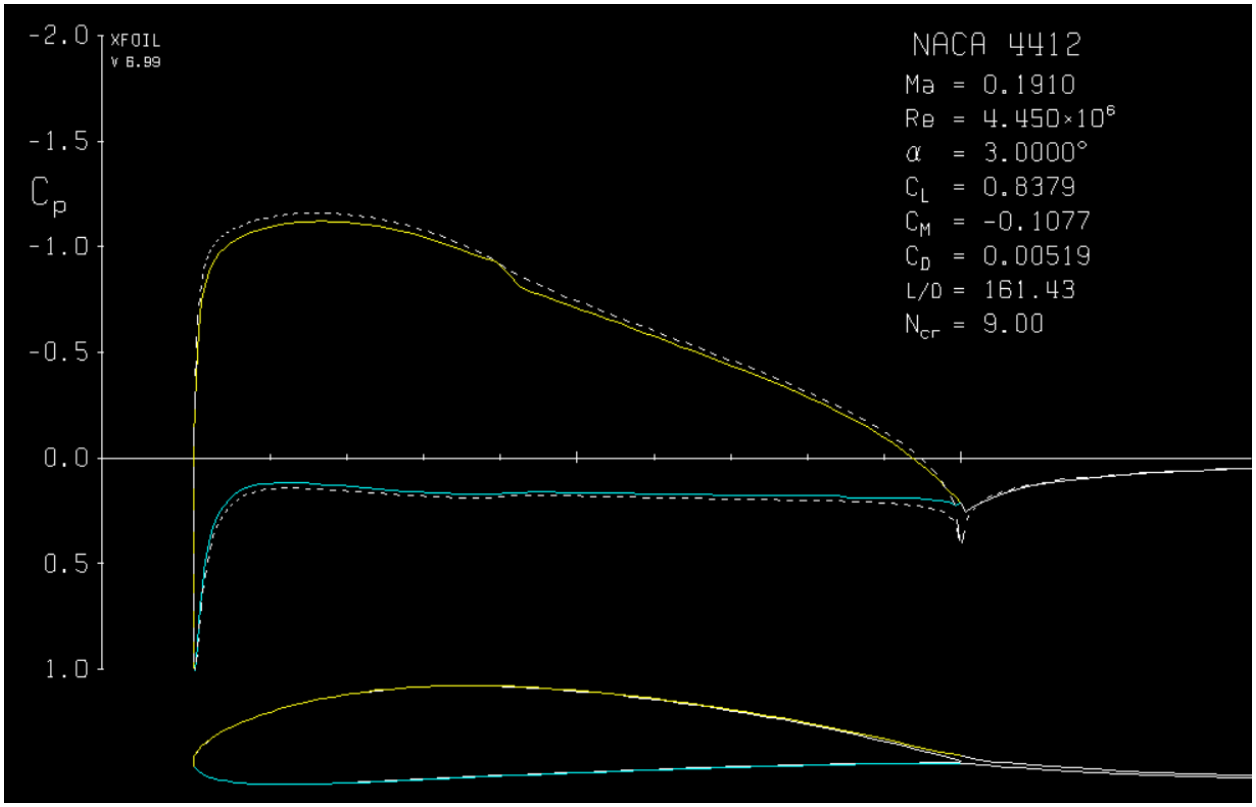
 # 11: Takeoff Configuration Simulation



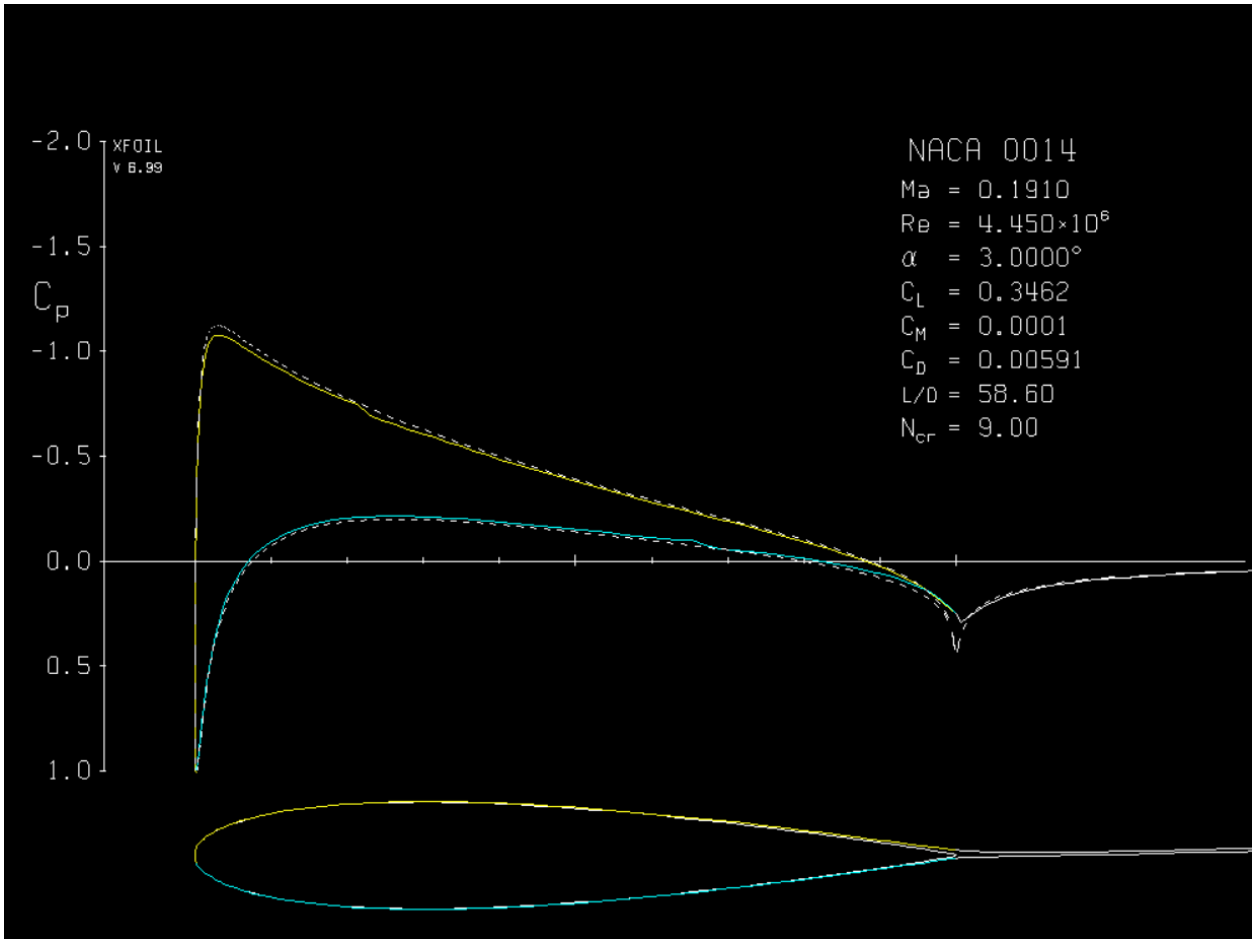
 # 12: Takeoff Configuration Simulation



 # 13: Takeoff Main Air Wing C_d Simulation:



 # 14: Takeoff Tail Airwing C_d Simulation



Source Code:

Preceding this page is the source code. These code files work together in a directory. With functions called across files.

#15 : Main Script: CalculatePerformance.mlx

Define Parameters

```
% Take off

% Drag



clear

clf

W_pounds.Wi = 255000;

W_pounds.Wp = 135896;

W_pounds.Wf = W_pounds.Wi - W_pounds.Wp;

Power_Horses = 4637 * 4;



W.i = 115666.054;

W.p = 61641.3887; % IN POUNDS

W.f = W.i - W.p;




Power = 3410000 *4; %K Watts

ThrustEff = .65;



% Aircraft Wing Param

c = 2; % Cord Length (Constant for all surfaces)

b = [10,10,4,4,4]; % legth of each half wing



Recrit = 10^6;
```

```
% Defining Fuselage Parameters
Fuselage.x1 = 0; % Starting x bound
Fuselage.x2 = 35.36; % Ending x bound
Fuselage.y1 = 0; % Starting y bound
Fuselage.y2 = 13.41; % Starting y bound

% Defining Conditions
%   Cruise
conditions.cruise.alt = 6705.6;
conditions.cruise.rho = 0.61003;
conditions.cruise.U_inf = 180;
conditions.cruise.v = 2.602e-5;
conditions.cruise.SoS = 313.5318;
conditions.cruise.Re = 6.9887e+6;
%   Sea Level
conditions.sea.alt = 0;
conditions.sea.rho = 1.2250; % Density at altitude
conditions.sea.U_inf= 65; % Velocity of travel
conditions.sea.v = 0.000014607; % Kinematic Viscosity
conditions.sea.SoS = 340.294;
conditions.sea.Re = 4.4499e+6;

%   Landing Conditions
conditions.landing.alt = 0;
```

```
conditions.landing.rho = 1.2250; % Density at altitude

conditions.landing.U_inf= 62.5265; % Velocity of travel

conditions.landing.v = 0.000014607; % Kinematic Viscosity

conditions.landing.SoS = 340.294;

conditions.landing.Re = 4.2805e+6;


% Wing and Elevator Object

%   Cruise

aerodynamics.cruise.C_l = .24;

aerodynamics.cruise.C_l_max = 1.0826;

aerodynamics.cruise.C_d = [0.00603,0.00603,0.00574,0.00574,0.00574];


%   Sea Level

aerodynamics.sea.C_l = 1.1; % Combine C_l

aerodynamics.sea.C_l_max  = 1.7389;

aerodynamics.sea.C_d = [0.00519,0.00519,0.00591,0.00591,0.00591]; % C_d per half wing


%   Landing Aerodynamics

aerodynamics.landing.C_l = 1.19; % Combine C_l

aerodynamics.landing.C_l_max  = 1.7389;

aerodynamics.landing.C_d = [0.01062,0.01062,0.00869,0.00869,0.00869]; % C_d per half wing
```

#16: Check for speeds for lift = weight

```
U_inf_match = Find_Lift_is_Weight(Fuselage, aerodynamics.sea, conditions.sea, c, b, W.i)
```

```
U_inf_match = 64.3216
```

#17: Takeoff Conditions Check

```
[Sea.Drag,Sea.Lift] = GetTotalAerodynamics(Fuselage,aerodynamics.sea,conditions.sea,c,b);

Sea.Lift_to_Weight = Sea.Lift / W.i;

Sea.Thrust = CalcThrust(conditions.sea,Power,ThrustEff);

Sea.Thrust_to_Drag = Sea.Thrust / Sea.Drag;

[Sea.IsStall,Sea.VStall] = CheckStall(conditions.sea,aerodynamics.sea,(b(1)*2*c),W.i);

Sea.U_inf = conditions.sea.U_inf;

Sea.Mach = Sea.U_inf / conditions.sea.SoS;

Sea.alfa = 3.0;

Sea.X_np = 4.68;

conditions.sea
```

ans = struct with fields:

alt: 0

rho: 1.2250

U_inf: 65

v: 1.4607e-05

SoS: 340.2940

Re: 4449900

```
Sea
```

Sea = struct with fields:

Drag: 3964.778590421519902633297776645 9

Lift: 1.1600e+05

Lift_to_Weight: 1.0029

Thrust: 136400

Thrust_to_Drag: 34.402929921365036509957052374718

IsStall: 0

VStall: 52.1054

U_inf: 65

Mach: 0.1910

alfa: 3

X_np: 4.6800

#18: Cruise Conditions Check

```
[Cruise.Drag,Cruise.Lift] = GetTotalAerodynamics(Fuselage,aerodynamics.cruise,conditions.cruise,c,b);
Cruise.Lift_to_Weight = Cruise.Lift / W.i;
Cruise.Thrust = CalcThrust(conditions.cruise,Power,ThrustEff);
Cruise.Thrust_to_Drag = Cruise.Thrust / Cruise.Drag;
[Cruise.IsStall,Cruise.VStall] = CheckStall(conditions.cruise,aerodynamics.cruise,(b(1)*2*c),W.i);
Cruise.U_inf = conditions.cruise.U_inf;
Cruise.Mach = Cruise.U_inf / conditions.cruise.SoS;
Cruise.alfa = -0.6297;
Cruise.X_np = 4.54;
conditions.cruise
```

ans = struct with fields:

alt: 6.7056e+03

rho: 0.6100

U_inf: 180

v: 2.6020e-05

SoS: 313.5318

Re: 6988700

```
Cruise
```

```
Cruise = struct with fields:

        Drag: 18043.494490612653361081463810794

        Lift: 1.1587e+05

        Lift_to_Weight: 1.0018

        Thrust: 4.9256e+04

        Thrust_to_Drag: 2.729823515127923493239777753653 1

        IsStall: 0

        VStall: 93.5790

        U_inf: 180

        Mach: 0.5741

        alfa: -0.6297

        X_np: 4.5400
```

#19: Landing Conditions Check

```
% A = 10

% Flaps = 15

% Speed = 1.2 Stall

[Landing.IsStall,Landing.VStall] =
CheckStall(conditions.landing,aerodynamics.landing,(b(1)*2*c),W.i);

conditions.landing.U_inf = (1.2 * Landing.VStall); % Setting Landing conditions to 1.2 times stall

Landing.U_inf = conditions.landing.U_inf;

[Landing.Drag,Landing.Lift] =
GetTotalAerodynamics(Fuselage,aerodynamics.landing,conditions.landing,c,b);

Landing.Lift_to_Weight = Landing.Lift / W.i;
```

```
Landing.Thrust = CalcThrust(conditions.landing,Power,ThrustEff);

Landing.Thrust_to_Drag = Landing.Thrust / Landing.Drag;

[Landing.IsStall,Landing.VStall] =
CheckStall(conditions.landing,aerodynamics.landing,(b(1)*2*c),W.i);

Landing.Mach = Landing.U_inf / conditions.landing.SoS;

Landing.alfa = 8;

Landing.X_np =4.74;

conditions.landing
```

ans = struct with fields:

alt: 0

rho: 1.2250

U_inf: 62.5265

v: 1.4607e-05

SoS: 340.2940

Re: 4280500

```
Landing
```

Landing = struct with fields:

IsStall: 0

VStall: 52.1054

U_inf: 62.5265

Drag: 5062.17014770311601808405545115 56

Lift: 1.1596e+05

Lift_to_Weight: 1.0025

Thrust: 1.4180e+05

Thrust_to_Drag: 28.01090422524201272421703338 4775

Mach: 0.1837

alfa: 8

X_np: 4.7400

#20: Range Check

```
Range_At_Cruise_in_miles = RangeCalc(W_pounds,Power_Horses,conditions.cruise)
```

Range_At_Cruise_in_miles = 2.2903e+03

#21: Function for Thrust and Drag: GetTotalAerodynamics.m

```
function [Total_Drag,Total_Lift] = GetTotalAerodynamics(Fuselage,aerodynamics,conditions,c,b)
  % Calls other functions to get total Drag and Lift
  Df_Fuselage = dragFuselage(Fuselage,conditions);
  Df_Aerodynamics = GetWingDrag(aerodynamics,conditions,c,b);
  Total_Drag = (Df_Fuselage + Df_Aerodynamics) ./ (sqrt(1- ( (conditions.U_inf./conditions.SoS) ).^2 ));
  Total_Lift = GetWingLift(aerodynamics,conditions,c,b) ./ (sqrt(1- ( (conditions.U_inf./conditions.SoS)
).^2 ));
end
function [Drag_Fuselage] = dragFuselage(o, cond)
  % This function calculates the drag of the Fuselage by integrating over
  % C_f. For these conditions we have found that the aircraft will be
  % pure turb
  %input: two objects, one for the part, one for the atm conditions
  syms x y
  % skin friction coefficients for laminar and turbulent conditions
  cf_lam =  0.664 * cond.v^0.5 / (cond.U_inf * x)^0.5;
  cf_turb =  0.0592 * cond.v^0.2 / (cond.U_inf*x)^0.2;

  % drag
  Drag_Fuselage = vpa(.5 * cond.rho * cond.U_inf^2 * ( int(int(cf_turb, x, o.x1, o.x2), y, o.y1, o.y2) ) , 4
);
end
function [Drag] = GetWingDrag(object,cond,c,b)
  % Simple function takes in the wing section C_D object and calculates the
  % total drag of the wing(s)
  D = (object.C_d .* 0.5 .* cond.rho .* (cond.U_inf.^2) .* c) .* b .* 2;
  Drag = sum(D);
end
function [Lift] = GetWingLift(object,cond,c,b)
  % Simple function takes in the wing section C_l object and calculates
  % the lift of the wing assuming only the surface area of the main wing
  Lift = 0.5 .* object.C_l .* (c .* b(1) .* 2) .* cond.rho .* (cond.U_inf.^2);
end
```

#22: Function to fit a U_infiniy for a matching Lift/Weight: Find_Lift_is_Weight.m

```
function U_inf_match = Find_Lift_is_Weight(Fuselage, aerodynamics, conditions, c, b, W)
  % Define U_inf range to check
  U_inf_values = linspace(0, 200, 200); % Adjust range as needed
  found = false;

  for U_inf = U_inf_values
     conditions.U_inf = U_inf; % Updating U_inf

     % Compute drag and lift with the new value
     [Drag, Lift] = GetTotalAerodynamics(Fuselage, aerodynamics, conditions, c, b);

     % Check if Lift is within 5% of W
     if abs(abs(W / Lift) - 1) < 0.05
        found = true;
        U_inf_match = U_inf;
        break;
     end
  end

  if ~found
     U_inf_match = NaN; % No match found
     return;
  end

  % Refinement to three decimal places using binary search
  lower_bound = U_inf_match - 0.1;
  upper_bound = U_inf_match + 0.1;

  while (upper_bound - lower_bound) > 1e-3
     U_inf_refined = (lower_bound + upper_bound) / 2;
     conditions.U_inf = U_inf_refined;

     % Compute refined Lift
     [Drag, Lift] = GetTotalAerodynamics(Fuselage, aerodynamics, conditions, c, b);

     if abs(abs(W / Lift) - 1) < 0.005  % Tighter tolerance
        U_inf_match = U_inf_refined;
        upper_bound = U_inf_refined;
     else
        lower_bound = U_inf_refined;
     end
  endend
```

#23: Function to calculate the thrust: CalcThrust.m

```
function Thrust = CalcThrust(conditions,power,ThrustEff)
  Thrust = ThrustEff .* power ./ conditions.U_inf;
end
```

#24: Function to Check for Stall and V_Stall: CheckStall.m

```
function [isStalling,Vstall] = CheckStall(cond,Aero, S, W)
% S: planform area
% W: weight of the plane
Vstall = sqrt((2*W)/(cond.rho*S*Aero.C_l_max));
% cruise_spd = sqrt((2*W)/(cond.rho*S*Aero.C_l));
if cond.U_inf < Vstall
  display('ALERT: AIRCRAFT IS STALLING AT THIS CRUISING SPEED')
  display('REDESIGN WING')
  isStalling = true;
else
  isStalling = false;
end
end
```

#25: Function to calculate range: RangeCalc.m

```
function [Range] = RangeCalc(W,Ps,conditions)
  %at cruise
  C = 0.46; %lbs/hp-h
  conditions.U_inf = conditions.U_inf .* 2.237;
  Range = (conditions.U_inf * W.Wi / (Ps*4*C) ) * log(W.Wi/W.Wf);
end
```

Main script for proofs and proof testing: Proofs.mlx
#26: XFOIL Proof

```
syms x

v = 2.602e-5;

Uinf = 178.816; %m/s

cf_turb = @(x) 0.0592*v^0.2/(Uinf*x)^0.2;

Cdwing = vpa((2/3.5)*int(cf_turb,x,0,3.5),4)
```

```
Cdwing =

0.004943
```

#27: Compressibility Proof

```
clc

clear



M = 0.3:0.1:0.8;

m = mfoil('naca','4412', 'npanel',199);



%incompressable flow over mach with PrandtlG scaling

for i = 1:1:6

        %set parameters

        m.setoper('alpha',2,'Re',10^6)

        % run the solver, plot the results

        m.solve;

        Cd_prandtlG(i) = m.post.cd/sqrt(1-M(i)^2);
```

```
end
```

SKIPPING MANY MFOIL LINES OF OUTPUT

Newton iteration 25, Rnorm = 1.20922e-08

transition on upper side at x=0.52349

alpha=2.00deg, cl=0.698250, cm=-0.103427, cdpi=0.000757, cd=0.006253, cdf=0.004351, cdp=0.001902

Newton iteration 26, Rnorm = 2.08871e-12

alpha=2.00deg, cl=0.698250, cm=-0.103427, cdpi=0.000757, cd=0.006253, cdf=0.004351, cdp=0.001902

```
display(Cd_prandtlG)
```

Cd_prandtlG = *1×6*

```
    0.0066  0.0068  0.0072  0.0078  0.0088  0.0104
```

```
%compressable flow over mach
for i = 1:1:6
    %set parameters
    m.setoper('alpha',2, 'Re',10^6, 'Ma',M(i));
    % run the solver, plot the results
    m.solve;
    Cd_Mfoil(i) = m.post.cd;
end
```

<<< Solving inviscid problem >>>

alpha=2.00deg, cl=0.799348, cm=-0.120765, cdpi=-0.002290, cd=0.000000, cdf=0.000000, cdp=0.000000

SKIPPING MANY LINES OF MFOIL OUTPUT

Newton iteration 11, Rnorm = 1.16673e-09

transition on lower side at x=0.33301

transition on upper side at x=0.42303

alpha=2.00deg, cl=1.037610, cm=-0.121100, cdpi=-0.013808, cd=0.013853, cdf=0.005376, cdp=0.008477

Newton iteration 12, Rnorm = 3.60500e-12

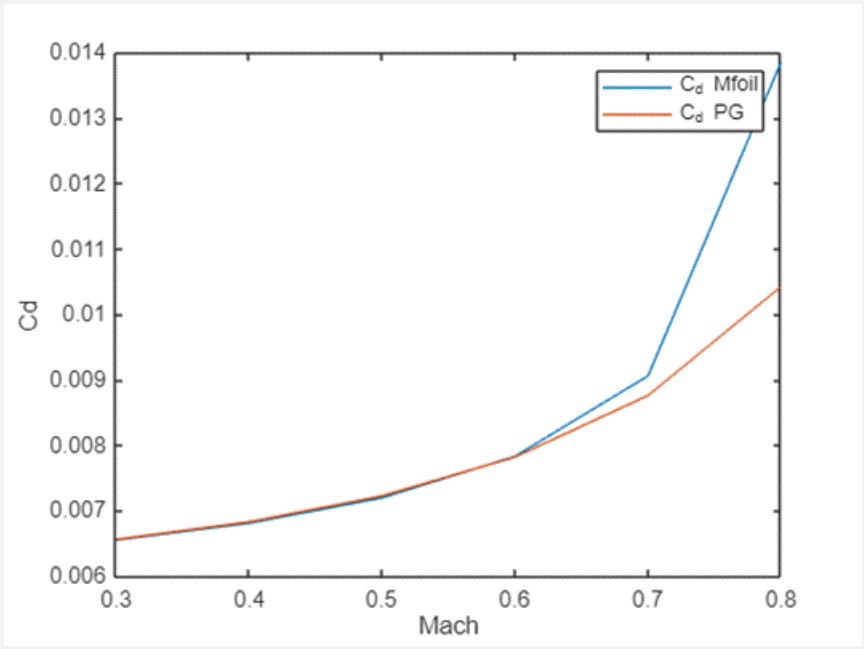
alpha=2.00deg, cl=1.037610, cm=-0.121100, cdpi=-0.013808, cd=0.013853, cdf=0.005376, cdp=0.008477

```
display(Cd_Mfoil)
```

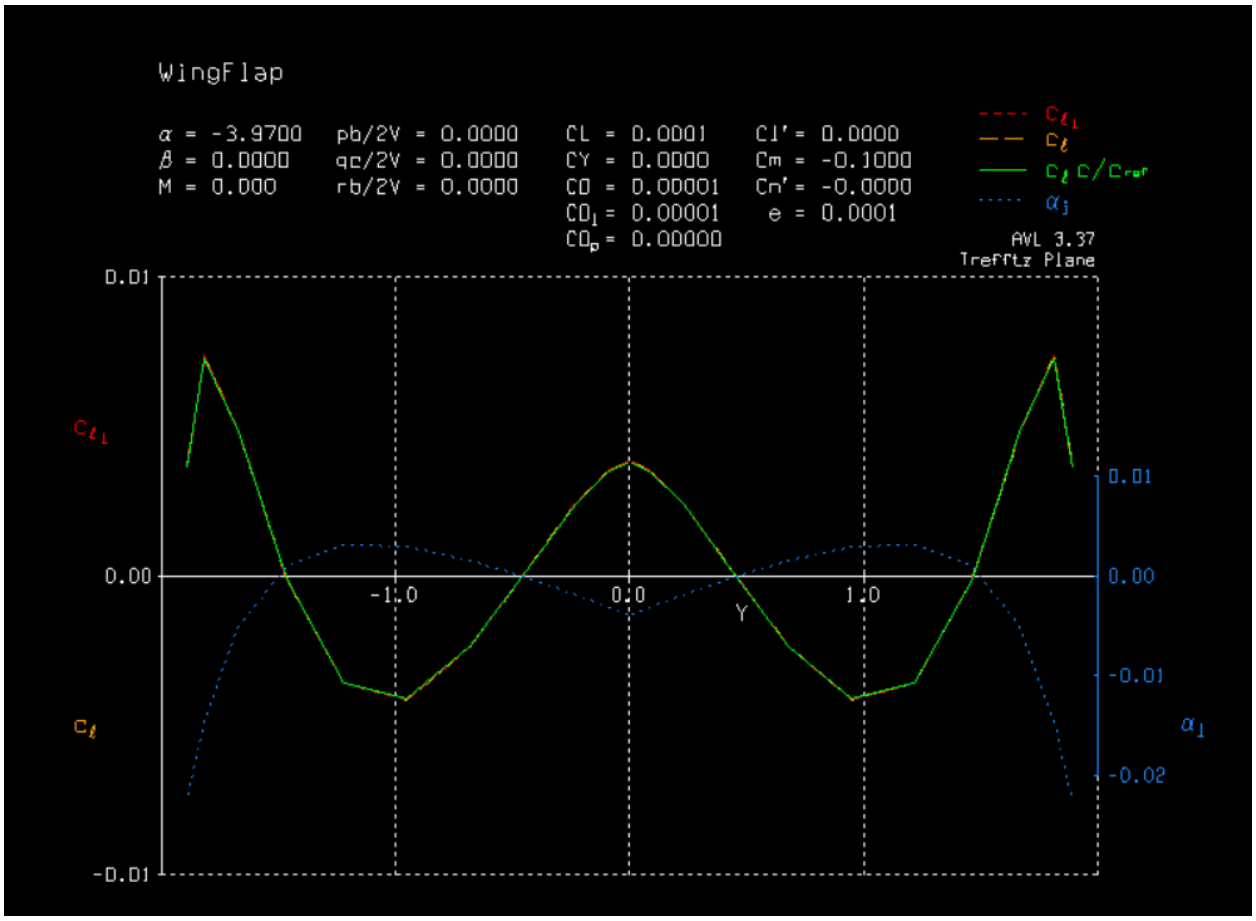
Cd_Mfoil = *1×6*

0.0065	0.0068	0.0072	0.0078	0.0091	0.0139

```
figure
plot(M,Cd_Mfoil)
hold on
plot(M,Cd_prandtlG)
hold off
ylabel('Cd')
xlabel('Mach')
axis([0.3 0.8 0.006 0.03])
legend("C_d Mfoil","C_d PG")
```


0.014
0.013
0.012
0.011
0.01
0.009
0.008
0.007
0.006
Cd
0.3
0.4
0.5
0.6
0.7
0.8
Mach
Cd Mfoil
Cd PG

#28a: AVL Proof Output



#28b: AVL Proof Code

Using HW 7 Work: AVL Proof

```
clear

clf



% CSV file info

csvFiles = {'NACA4414.csv','NACA1414.csv','re6.csv','re7.csv'};

scriptDir = fileparts(matlab.desktop.editor.getActiveFilename);



% Loop through each CSV file and import the data
```

```
for i = 1:numel(csvFiles)
        % Full path of the current CSV file (assuming it's in the same directory)
        csvFilePath = fullfile(scriptDir, csvFiles{i});

        % Read the CSV file as a table
        T = readtable(csvFilePath);

        % Get the variable names from the first row
        varNames = T.Properties.VariableNames;

        % Loop through each column and create variables in the workspace
        for j = 1:numel(varNames)
        % Extract the data, excluding the first row
        data = T.(varNames{j})(1:end);

        % Assign each column to a variable with the corresponding name
        assignin('base', varNames{j}, data);
        end
end
```

Setting Constants

```
Vinf = 1;
Vinf = 1;
rho = 1;
```

```
% % % PART _ % % %
%  some code
% % % PART _ % % %


c = (max(NACA4414_X) - min(NACA4414_X));


ZeroLiftAlfa = lift(NACA4414_X,NACA4414_Z,-3.97,c,Vinf,rho,false,false)
```

```
ZeroLiftAlfa = 5.3319e-04
```

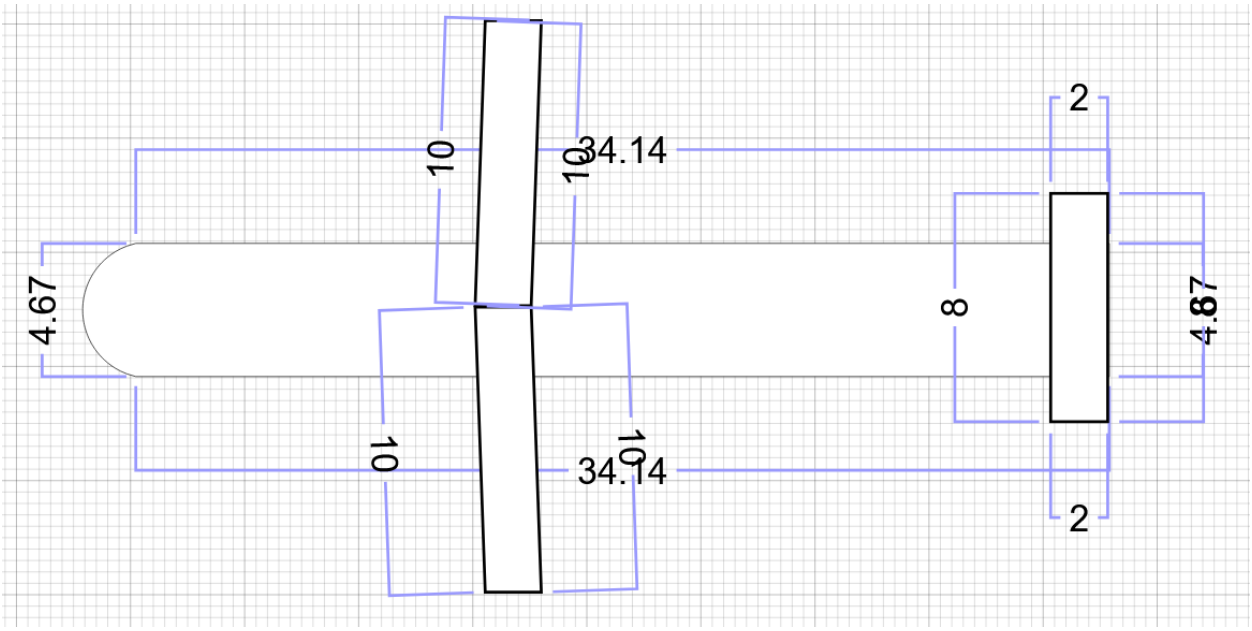
```
function [Lift_Coef] = lift(x,z,alpha,c,Vinf,rho,plooot,PartA)
% Setting Variables
N = length(x) - 1;
xj = x;
zj = z;
ni = ones(N,2);
A = zeros(N,N);
F = zeros(N,1);


% Setting control points as the mid point between each given node. Number
% of Nodes - 1 = Number of Control Points
for i=1:N
        xi(i) = (xj(i) + xj(i+1)) ./ 2;
        zi(i) = (zj(i) + zj(i+1)) ./ 2;
end
```

```
% Getting the Normal Vectors for use later
for j=1:N
        dx(j) = xj(j+1) - xj(j);
        dz(j) = zj(j+1) - zj(j);

        len = sqrt((dx(j)^2) + (dz(j)^2));
        ni(j,:) = [-dz(j)/len , dx(j)/len];
end

% Setting the Influence Coefficients (A) and the vortices
for i=1:N
        for j=1:(N+1)
        dx = xi(i) - xj(j);
        dz = zi(i) - zj(j);
        r2 = dx .* dx + dz .* dz;

        A(i, j ) = 1 ./ (2 * pi * r2) .* (dz *  ni (i,1) - dx * ni(i ,2));
        end
F(i) = -Vinf .* ( cosd(alpha) .* ni(i,1) + sind(alpha) .* ni(i ,2));
end

% Forcing the Kuda Condition
A(N+1,1) = 1;
A(N+1,N+1) = 1;
```

```
F(N+1) = 0;


% % % PART A % % %
% solving the system
Gamma = A\F;
if (PartA)
        Gamma
end
% % % PART A % % %


% % % PART B % % %
% Calculating the Lift and Lift Coeff
L = rho * Vinf * sum(Gamma)  * c;
Lift_Coef = L / (0.5 * rho * Vinf.^2 * c);
% % % PART B % % %


% % % PART D % % %
% Boolian check for whether or not to plot the streamlines for the Bonus
% Question
if (plooot)
        figure(1); clf ;
        [Y,X] = meshgrid(linspace(-0.5, 0.5, 51),linspace(-0.5, 1.5, 101));
        Z = X + Y*1i;
        F = Z*Vinf*(cosd(alpha) - 1i*sind(alpha));
```

```
        for j = 1:N, F = F + Gamma(j)*1i/(2*pi) * log(Z - xj(j) - zj(j)*1i); end
        Psi = imag(F);
        contour(X,Y,Psi,25); hold on;
        plot(xj, zj , 'k-', 'linewidth' , 2); axis equal;
        u = ni(:,1);
        v = ni(:,2);
        xi = xi.';
        zi = zi.';
        %quiver(xi,zi,u,v % BugChecking for Normal Vectors
        hold off
end
% % % PART D % % %
end
```

#29: Dimensional Display of Aircraft (1)



#30: Dimensional Display of Aircraft (2)